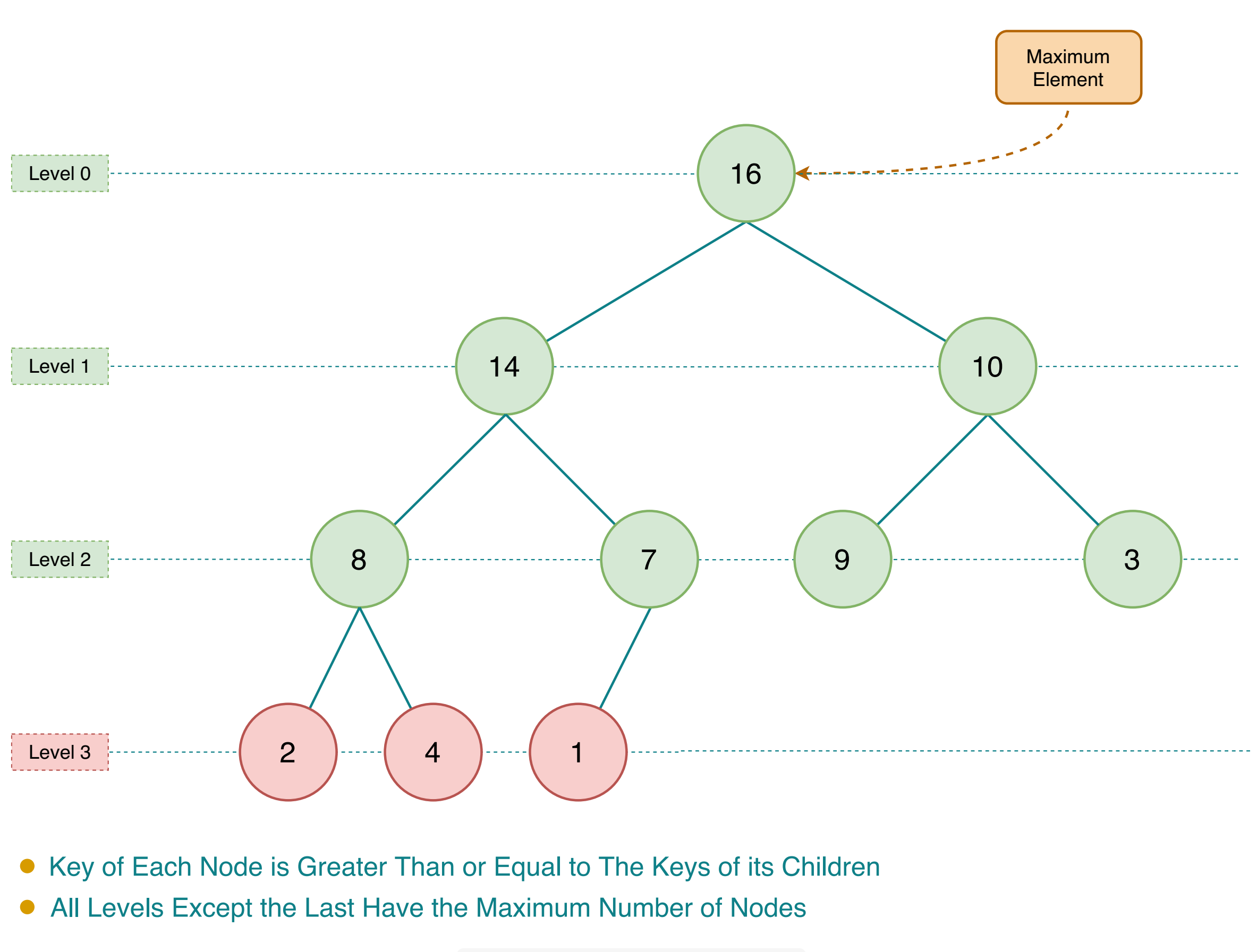
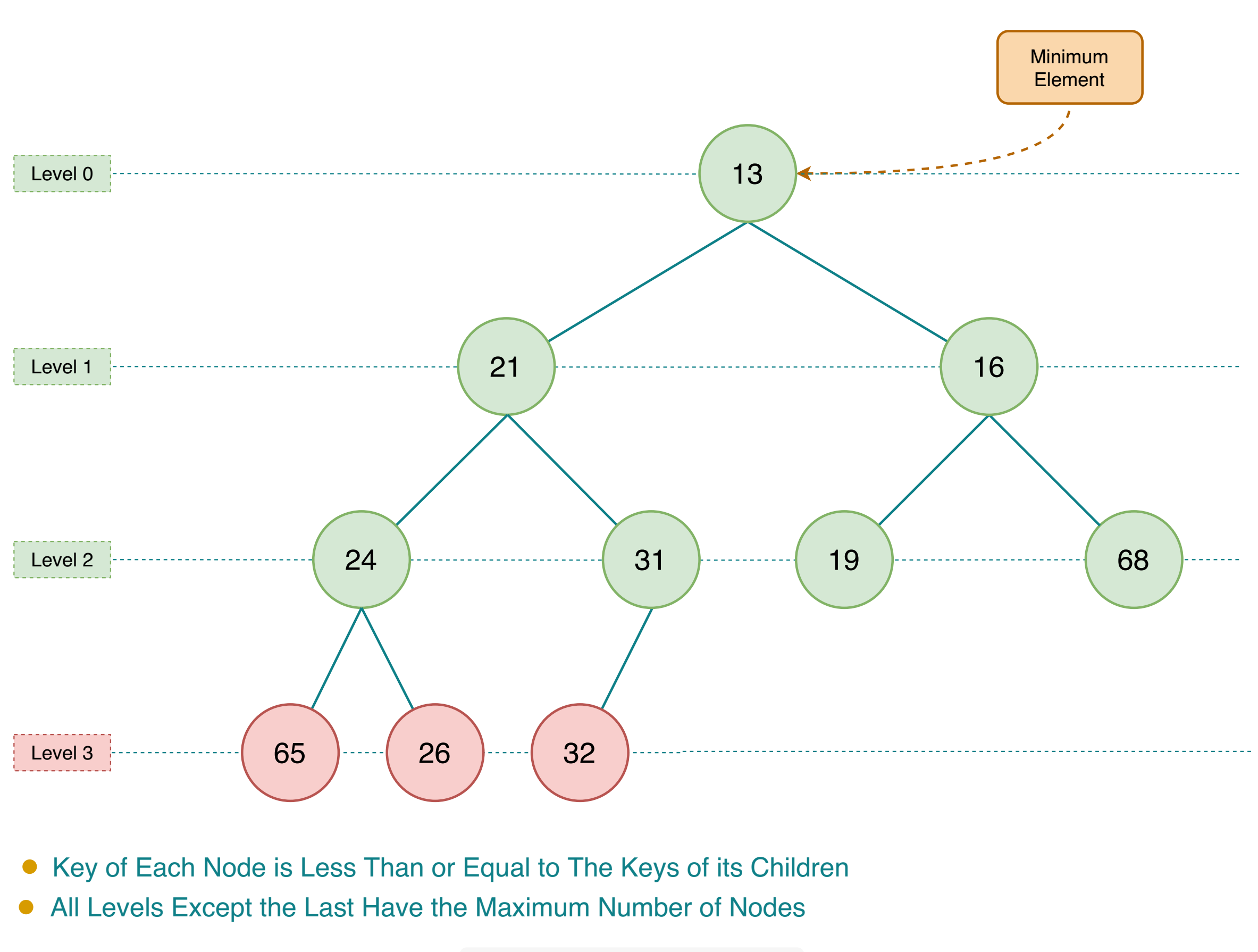
Introduction to Heap

Heaps are a specialized data structure and are based on binary trees. They are of the following two types:

- In a **max heap**, every parent node has a value **bigger** than or equal to its children.
- In a **min heap**, every parent node has a value **smaller** than or equal to its children.


Maximum Element
Level 0
Level 1
Level 2
Level 3
16
14
10
8
7
9
3
2
4
1
Key of Each Node is Greater Than or Equal to The Keys of its Children
All Levels Except the Last Have the Maximum Number of Nodes

An Example of MaxHeap.


Minimum Element
Level 0
Level 1
Level 2
Level 3
13
21
16
24
31
19
68
65
26
32
Key of Each Node is Less Than or Equal to The Keys of its Children
All Levels Except the Last Have the Maximum Number of Nodes

An Example of MinHeap.

This ordering property allows heaps to efficiently support operations like finding the maximum or minimum element.

Some key characteristics of heaps:

- Heaps are commonly implemented as binary heaps, which are binary trees represented using arrays.
- The time required for insertion and deletion is O(log n). Max/Min can be found in O(1).
- Heaps allow efficient implementation of priority queues, removing elements based on priority instead of insertion order.
- The heapify operation efficiently converts an array into a heap in O(n) time.
- Heaps are employed in heap sorting by continually extracting components in the correct sequence, which takes O(n log n) time.
- Applications include graph algorithms like Dijkstra's, scheduling algorithms, and sorting.

So, heaps are tree structures that enable quick access to the minimum or maximum value and allow priority-based ordering. Their speed and flexibility make them useful data structures for many algorithms. One of the ways to implement heaps is by implementing them as binary heaps.

2. Binary Heaps

Binary heaps are a common implementation of heaps as binary trees. They have the following key properties:

- Structure of Heap
    - Completeness - All levels of the tree are completely filled, except the last level, filled from left to right.
    - Shape - The binary tree must have the correct shape and be a full binary tree, with the last level having the fewest nodes of all the levels.
- Heap order property:
    - Max heap - The key of a node is greater than or equal to the keys of its descendants.
    - Min heap - The key of a node is less than or equal to the keys of its descendants.

Binary heaps can effectively enable determining the minimum or maximum thanks to this order property, which can be done in O(1) time by examining the root node. The structure of a heap is a quality that combines completeness and shape.

It is essential to maintain the heap ordering property to ensure the proper functioning of insertion and deletion operations. To keep the heap invariant, newly added elements should either "bubble up" or "bubble down."

3. Basic Operations of Heap

Insert Element

We can insert a new item in the heap using an iterative or recursive approach.

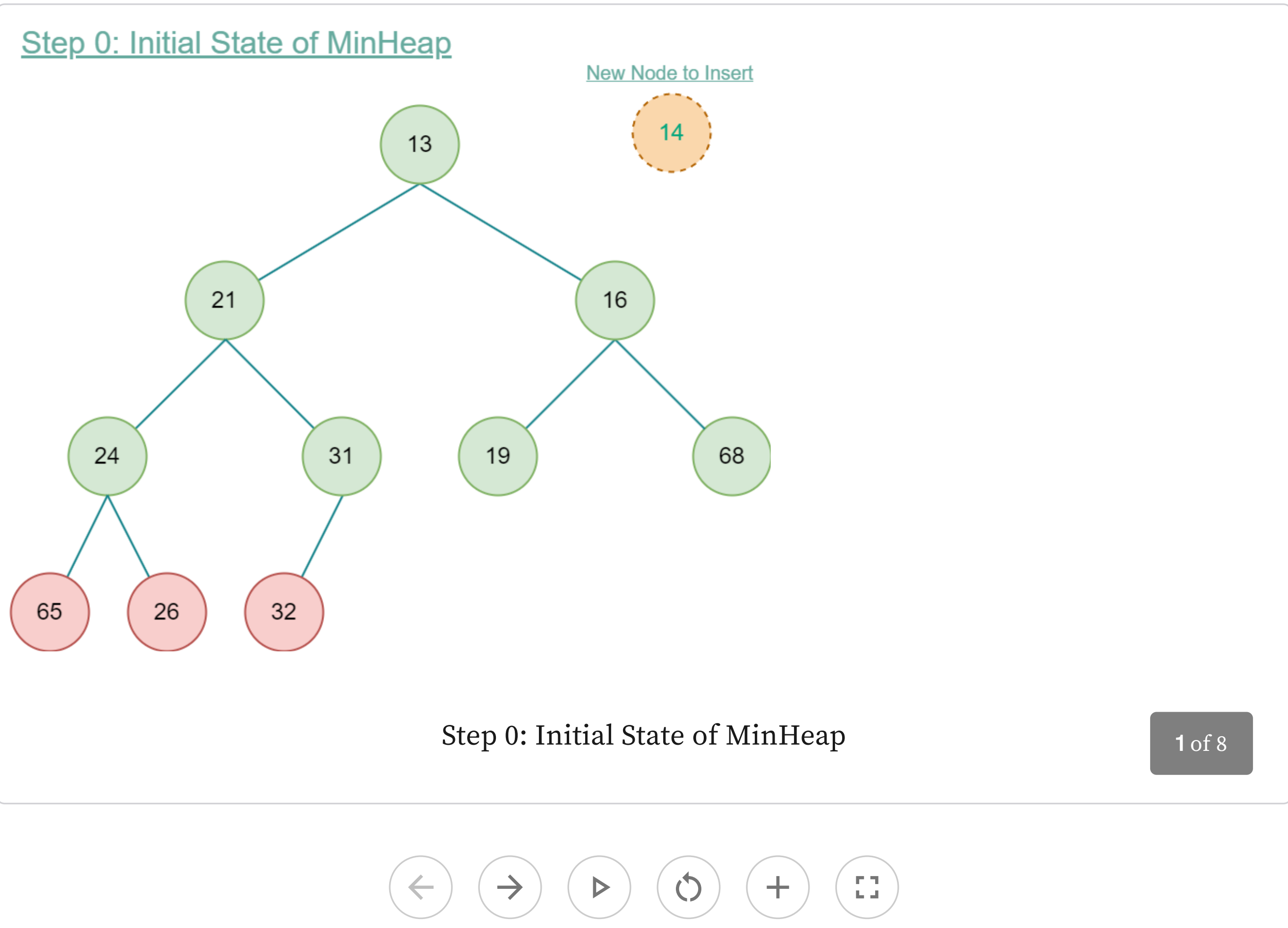
Iterative Approach

For iterative approach we can follow given steps:

1. Append the new node at the end of the array.
2. Initialize index to the new item's index
3. Loop while index is not at the root:
    1. Calculate the parent's index
    2. Swap if the parent has the greater key.
    3. Set index to the parent's index
4. Exit loop once heap property is satisfied

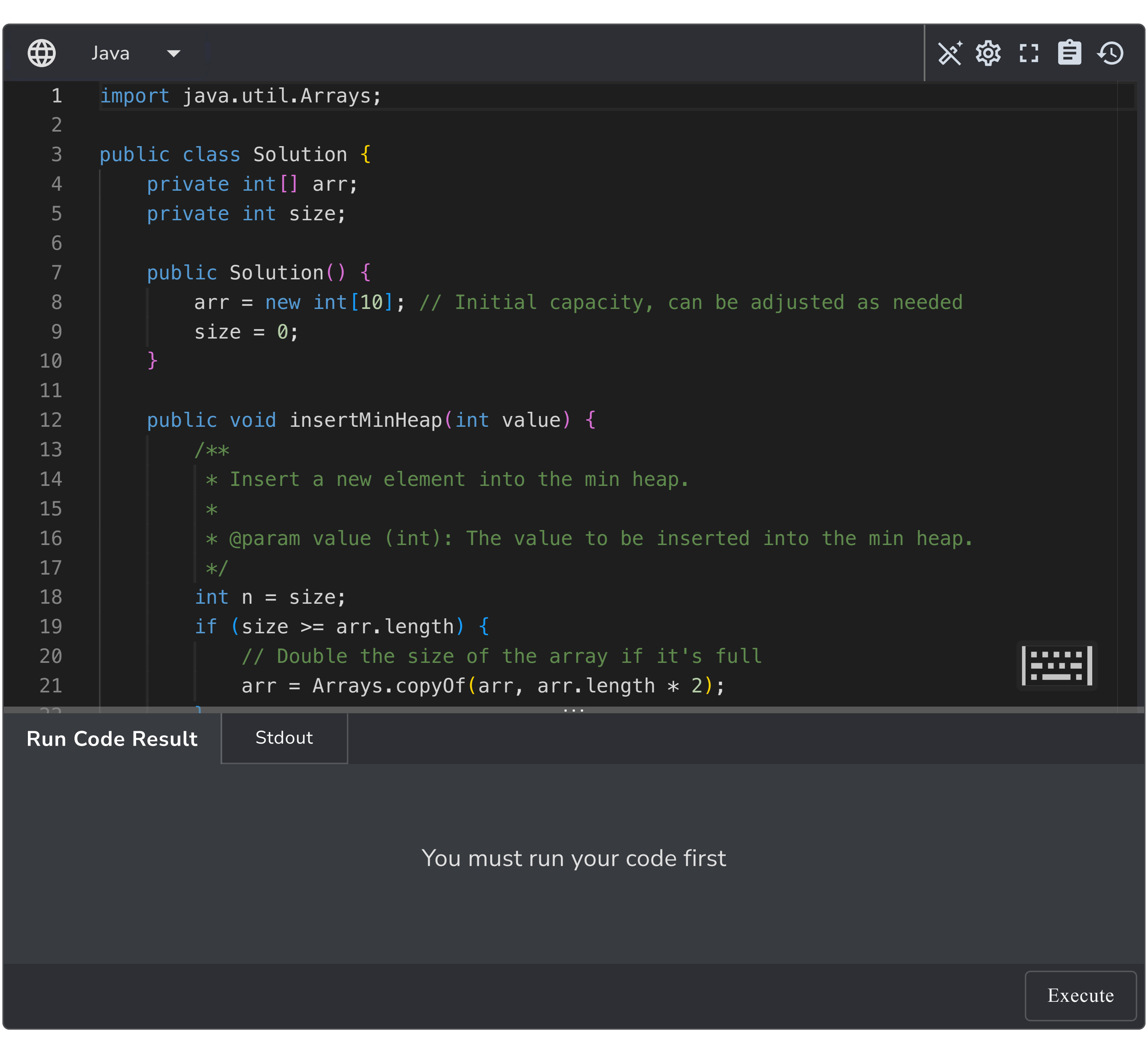
```
function insertMinHeap(arr, value):
    n = length of arr
    append value to arr
    index = n
    while index > 0 and value < arr[(index - 1) / 2]:
        arr[index] = arr[(index - 1) / 2]
        index= (index - 1) / 2
    arr[index] = value
```

This iteratively "bubbles up" the new item until the heap invariant is maintained. The loop version avoids recursion overhead. Insertion will take O(log n) time. The insertion of a new node in a MinHeap is visualized below.



Step 0: Initial State of MinHeap

Code for inserting a new node in MinHeap in different languages is given below.



```java
import java.util.Arrays;

public class Solution {
    private int[] arr;
    private int size;

    public Solution() {
        arr = new int[10]; // Initial capacity, can be adjusted as needed
        size = 0;
    }

    public void insertMinHeap(int value) {
        /**
         * Insert a new element into the min heap.
         *
         * @param value (int): The value to be inserted into the min heap.
         */
        int n = size;
        if (size >= arr.length) {
            // Double the size of the array if it's full
            arr = Arrays.copyOf(arr, arr.length * 2);
```

You must run your code first

Recursive Approach

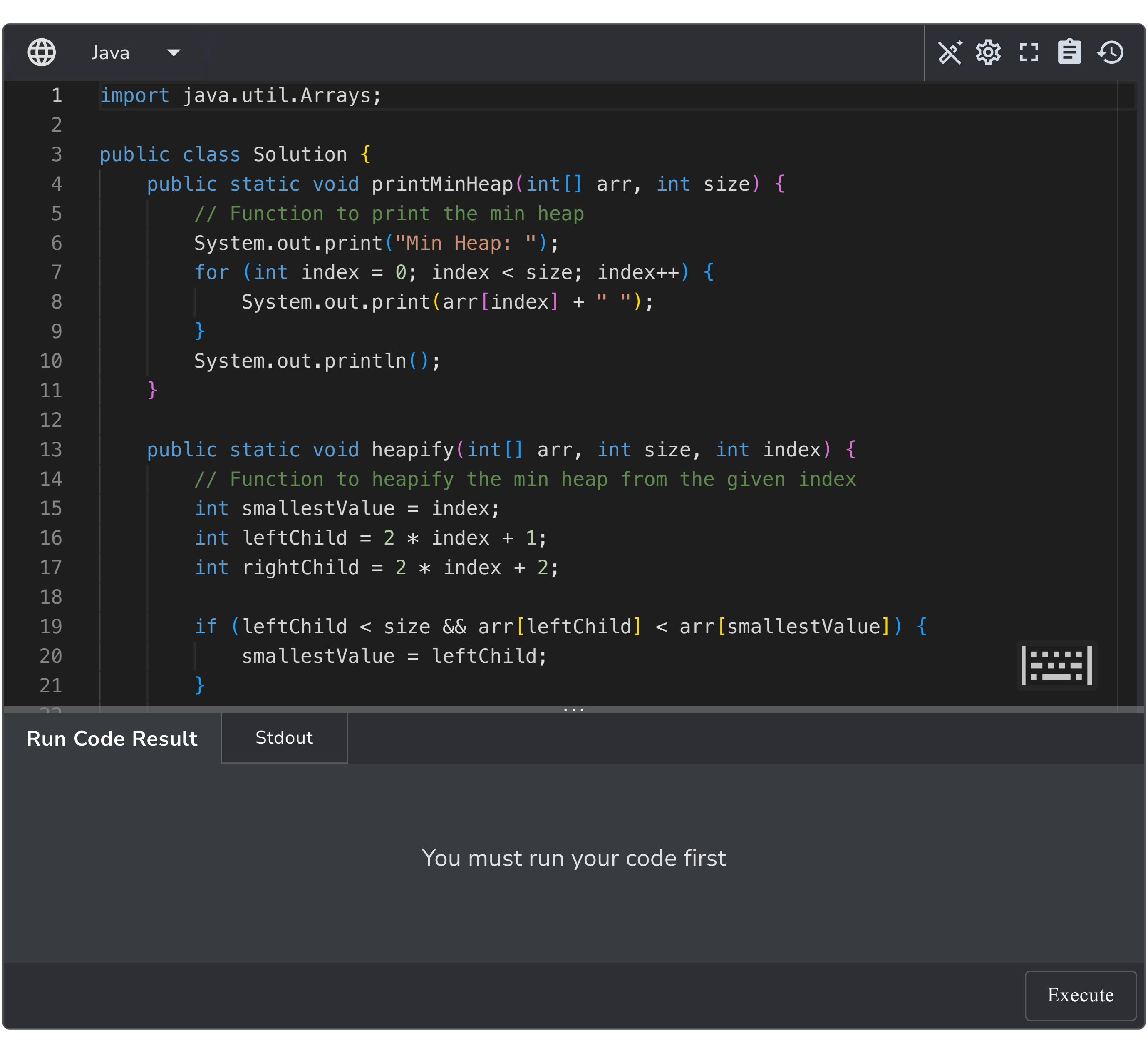
For recursive approach we can follow given steps:

1. Append new item to end of heap array
2. Call recursive helper to heapify up:
    1. If index 0, return (base case)
    2. Get parent index
    3. If parent > item, swap them
    4. Call heapifyUp recursively on the parent index.

```
function insert_MinHeap(arr, val):
    // Add the new value to heap
    arr.append(val)
    size = length(arr)

    // Call the recursive helper function to fix the min heap property
    heapify_up(arr, size - 1)

function heapify_up(arr, index):
    if index == 0:
        // Reached the root of the heap, stop recursion
        return

    parent_index = floor((index - 1) / 2)

    // If  less than parent, swap them
    if arr[parent_index] > arr[index]:
        swap(arr[parent_index], arr[index])
        // Continue heapifying up from the parent node
        heapify_up(arr, parent_index)
```

This moves the item up the tree while exchanging it till the heap property is met. The item is "bubbled up" by recursive calls in order to keep the heap invariant. Where n is the number of nodes, insertion takes O(log n) time.

Code for inserting a new node in MinHeap in different languages is given below.

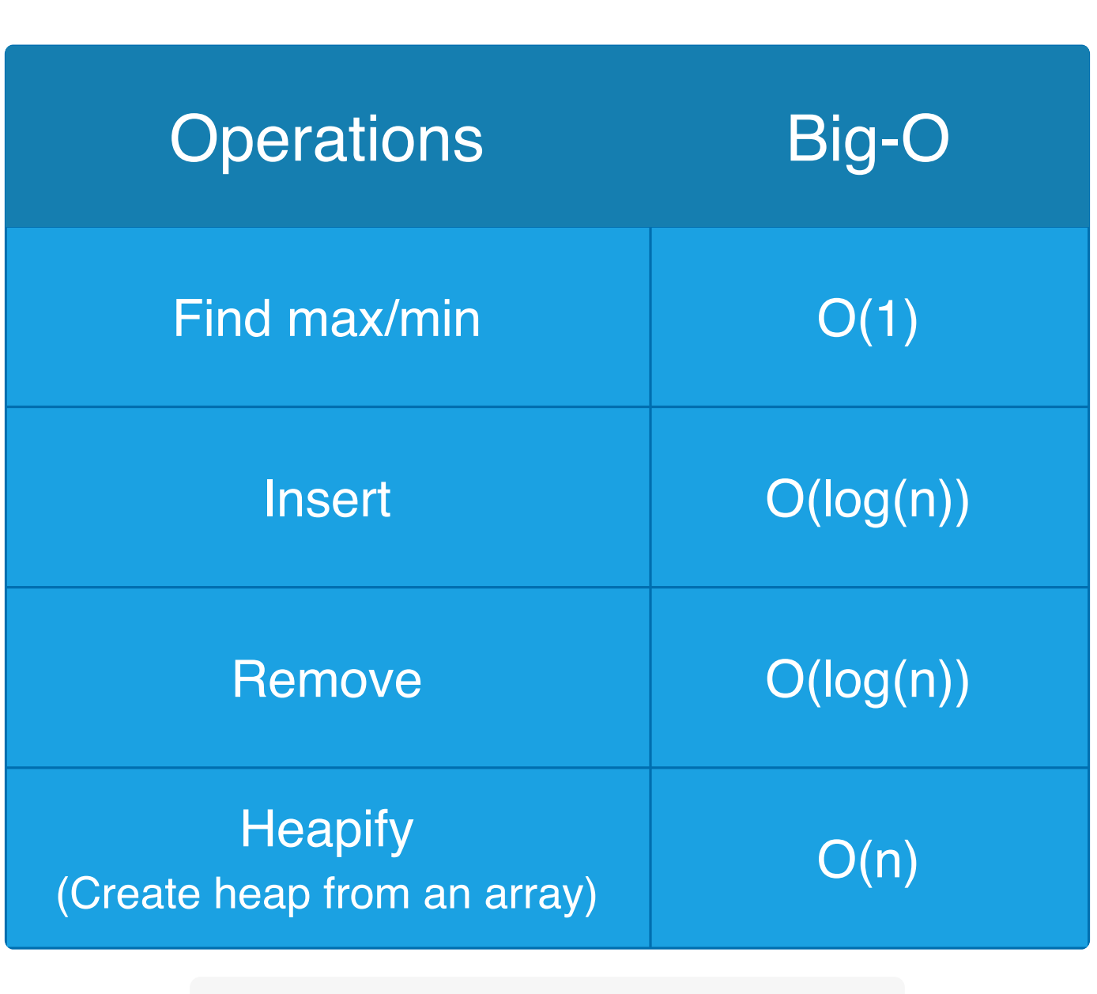


```java
import java.util.Arrays;

public class Solution {
    public static void printMinHeap(int[] arr, int size) {
        // Function to print the min heap
        System.out.print("Min Heap: ");
        for (int index = 0; index < size; index++) {
            System.out.print(arr[index] + " ");
        }
        System.out.println();
    }

    public static void heapify(int[] arr, int size, int index) {
        // Function to heapify the min heap from the given index
        int smallestValue = index;
        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;

        if (leftChild < size && arr[leftChild] < arr[smallestValue]) {
            smallestValue = leftChild;
        }
```

You must run your code first

Time Complexity

Here is the summary of big-O for different heap operations:



Operations	Big-O
Find max/min	O(1)
Insert	O(log(n))
Remove	O(log(n))
Heapify (Create heap from an array)	O(n)

Time Complexity of Heap Functions

Implementation

Here is how different languages have implemented Heap:

Language	API
C++	std::priority_queue
Java	java.util.PriorityQueue
Python	heapq
JavaScript	N/A

← Previous
Solution: Longest Substring Without Repeating Characters (Medium)

Next →
Take Gifts From The Richest Pile(Easy)

☐ Mark as Completed